HOWTO

CONTINUOUS PERFORMANCE ENGINEERING

Henrik Ingo

Fosdem 2026

NYRKIÖ

POLL

Continuous Performance Engineering maturity levels

1. We don't really do benchmarking
2. Benchmarks yes, continuous no
3. Some benchmarks run in CI

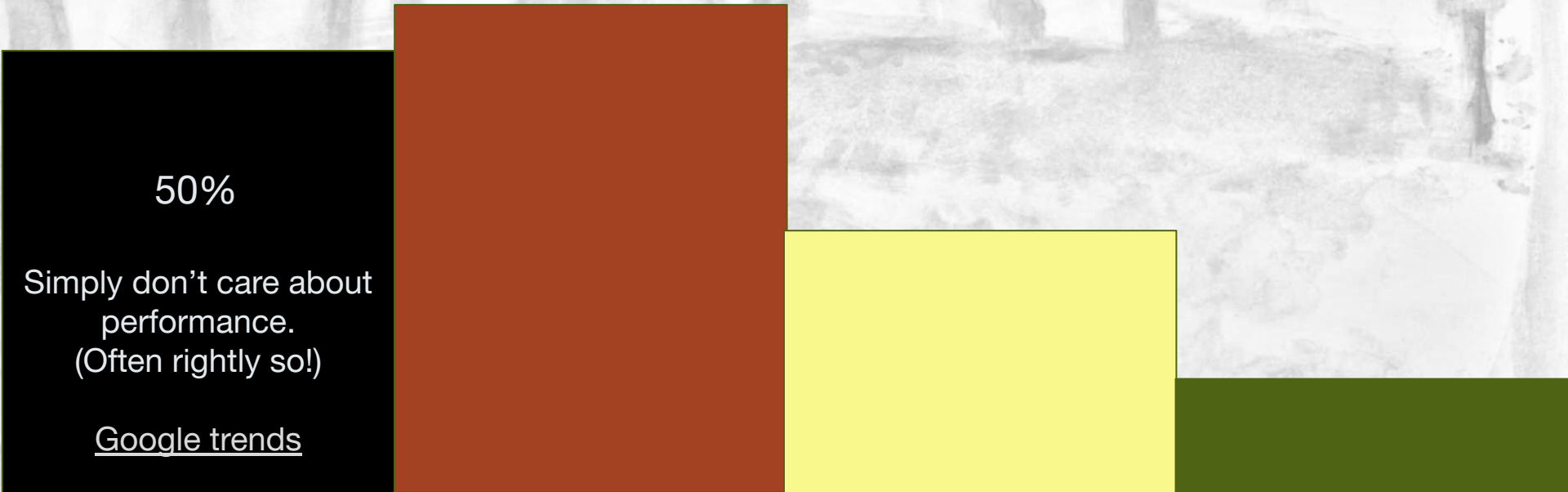
   …but we ignore the results
4. Benchmarks in CI,

   Daily or more often,

   Actionable results,

   Regressions fixed within weeks

Typical distribution of answers (normal developers)



TYPICAL DISTRIBUTION OF ANSWERS (NORMAL DEVELOPERS)

CONTINUOUS INTEGRATION

CONTINUOUS TESTING

CONTINUOUS DEPLOYMENT

DEVOPS

LEFT SHIFTING

ITERATIONS

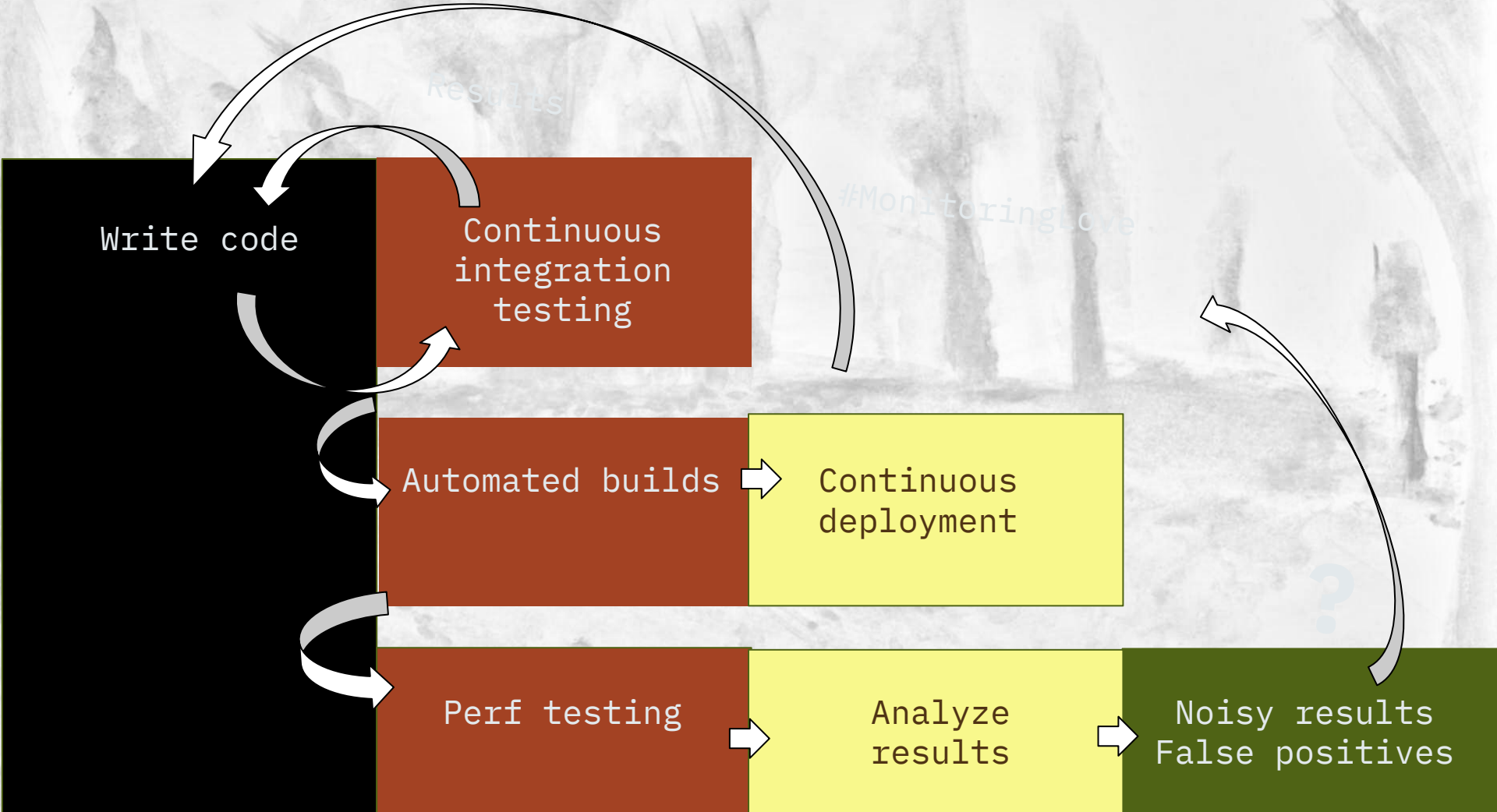
NYRKIÖ

CONTINUOUS PERFORMANCE ENGINEERING?

Somewhere in that evolution, they forgot to bring the performance engineer

Velociraptor, the fastest dinosaur (wikipedia)

NYRKIÖ


Write code
Continuous integration testing
Automated builds
Continuous deployment
Perf testing
Analyze results
Noisy results
False positives

WHY WAS PERFORMANCE ENGINEERING LEFT IN 1999?

Deploying is necessary - benchmarking is optional

Performance Engineers busy fixing prod / customer problems

….and maybe enjoying it?

Math is hard

Relevant tuning is unintuitive, not well known

CONTINUOUS PERFORMANCE ENGINEERING

NYRKIÖ

CONTINUOUS PERFORMANCE ENGINEERING IS POSSIBLE

NYRKIÖ

CONTINUOUS PERFORMANCE ENGINEERING

~~IS POSSIBLE~~

IS ACHIEVABLE!

NYRKIÖ

I WILL TELL YOU **HOW**

IF **YOU** PROMISE

TELL EVERYONE ELSE?

NYRKIÖ

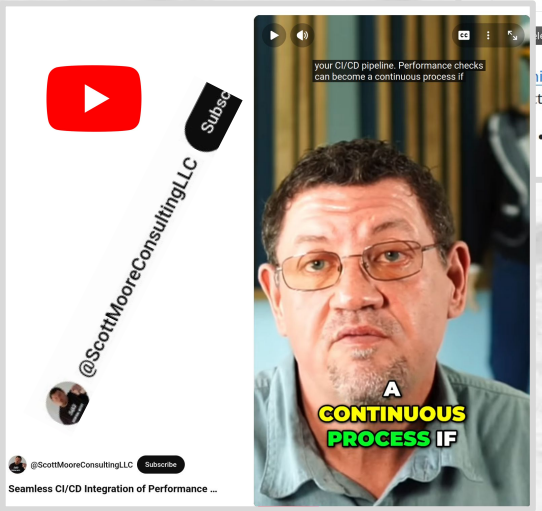
POLL

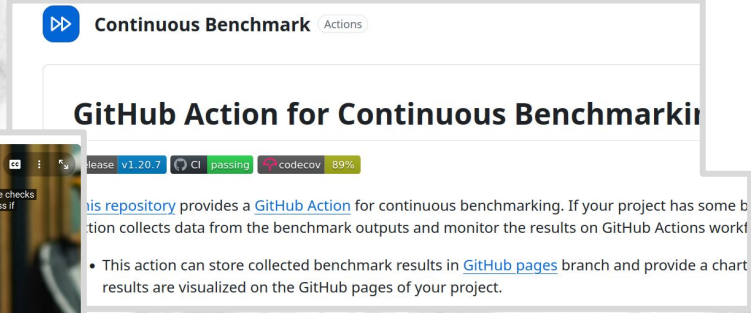
The community is so new, we don't even have consensus for what to call it?

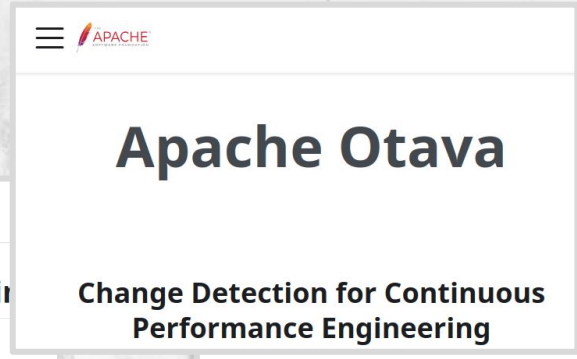
1. Continuous Performance Engineering
2. Continuous Benchmarking
3. DevPerfOps
4. Something else?
5. How should I know, I'm hearing about this for the first time..,.

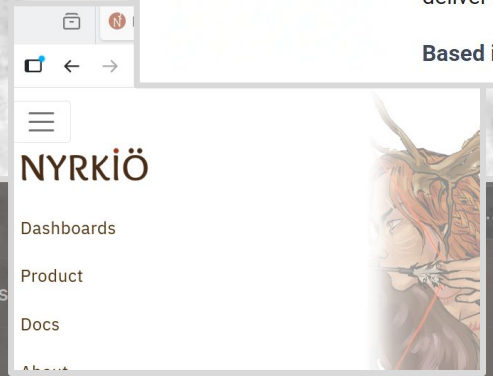
NYRKIÖ

WHO ELSE TALKS ABOUT THIS?









WHO ELSE IS ALREADY DOING IT?

NETFLIX

MongoDB

DATASTAX

CONFLUENT

TigerBeetle

dremio

Red Hat OpenShift

Kieker

TURSO
SQLite for Production

TARANTOOL

NYRKIÖ

AGENDA

1. Benchmarking
   - Make it Continuous
2. Change Point Detection
   - Math & Science
3. Minimizing noise in the Benchmarks
   - Assume nothing.
   - Measure everything.

NYRKIÖ

BENCHMARKING PROCESS AND TOOLS

NYRKIÖ

ON BENCHMARK TOOLS & DESIGN

Previous talk by Kemal Akkoyun & Augusto de Oliveira.

But in general…

- at this point each language has its own standard framework:
  - Java & JMH
  - Python & pytest-benchmark
  - etc…

Frameworks to run fully, end to end automated distributed benchmarks exist. Personally I believe we will see new innovation coming here. (k8s)

Core principles (part 1)

Sharing* is caring

Writing** is powerful

When writing, including at at least one graph is mandatory

*) Benchmark results, findings

**) Or present at conferences

NYRKIÖ

Core principles (for continuous benchmark design)

You get what you measure

Basic scientific process: Hypothesize -> Configure -> Test -> Analyze -> GOTO 1

Only change one thing per iteration

Core Principles (For Continuous Benchmark Design)

Benchmarks should be represent **realistic customer workloads**

However, that can take weeks, or months, just for a single benchmark.
Not agile!

One simple benchmark running in CI is more valuable than the perfect

Simple benchmarks are easier to analyze and reason about

For Continuous Benchmarking, having even one benchmark running in CI, allows you to start tracking it's history with graphs and change point detection!

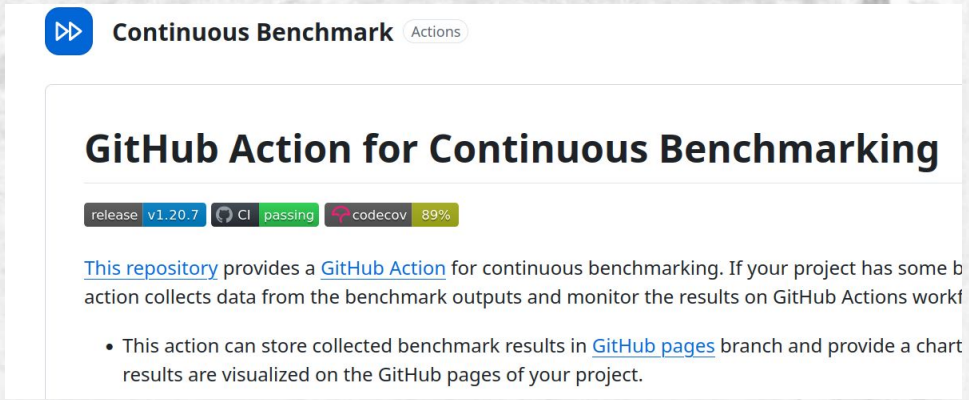
GITHUB-ACTION-BENCHMARK

Use in your workflows immediately after the benchmark step

Supports output from all major frameworks

Stores result history in your github repo.

Threshold based alerts.
Default threshold = 100% (2x)



NYRKIÖ

GITHUB-ACTION-BENCHMARK

Use in your workflows immediately after the benchmark step

Supports output from all major frameworks

Stores result history in your github repo.

Threshold based alerts.
Default threshold = 100% (2x)

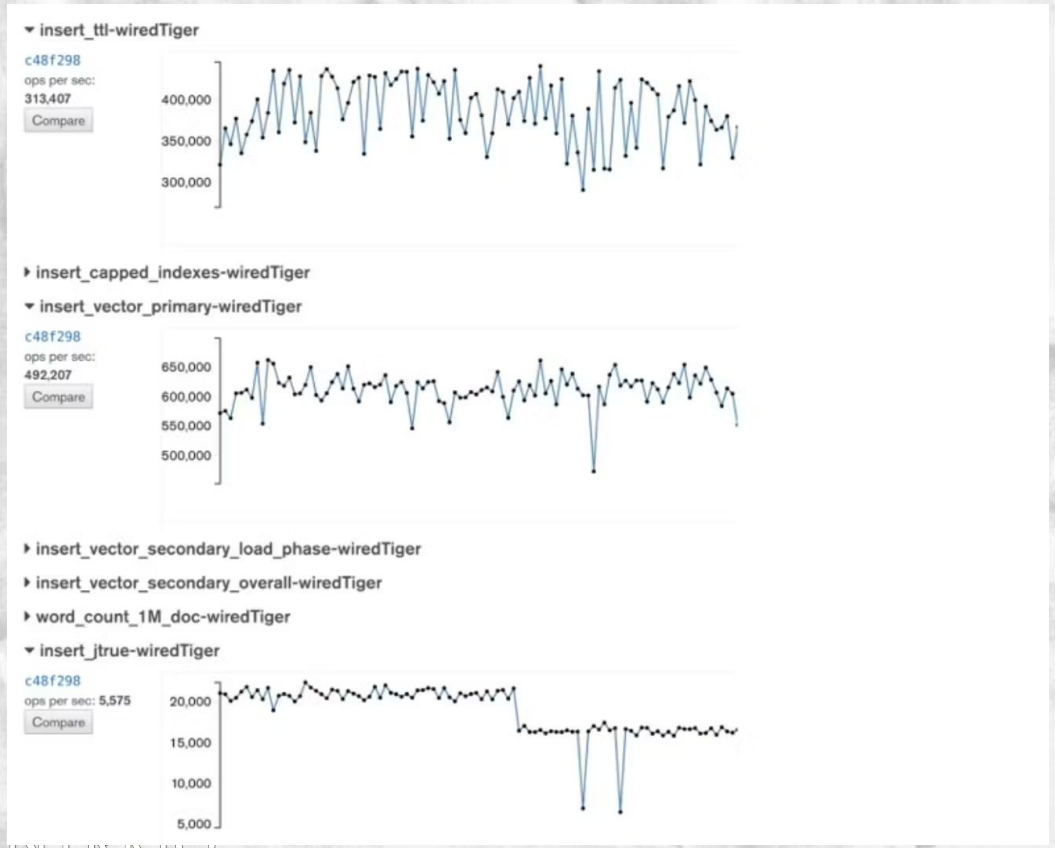
```yaml
jobs:
  benchmark:
    name: Performance regression check
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-go@v4
        with:
          go-version: "stable"
      # Run benchmark with `go test -bench` and stores the output to a file
      - name: Run benchmark
        run: go test -bench 'BenchmarkFib' | tee output.txt

      - name: Analyze benchmark results with Nyrkiö
        uses: nyrkio/change-detection@v2
        with:
          tool: 'go'
          output-file-path: output.txt
```

NYRKIÖ

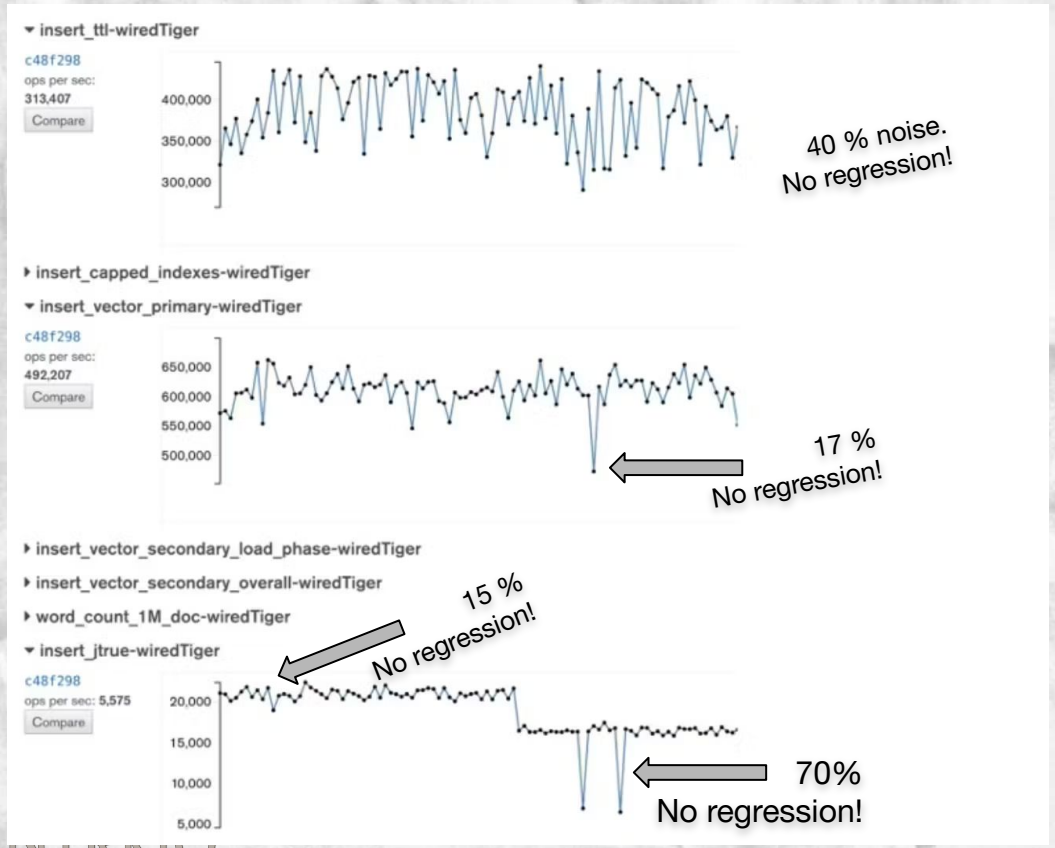
CHANGE POINT DETECTION

NYRKIÖ

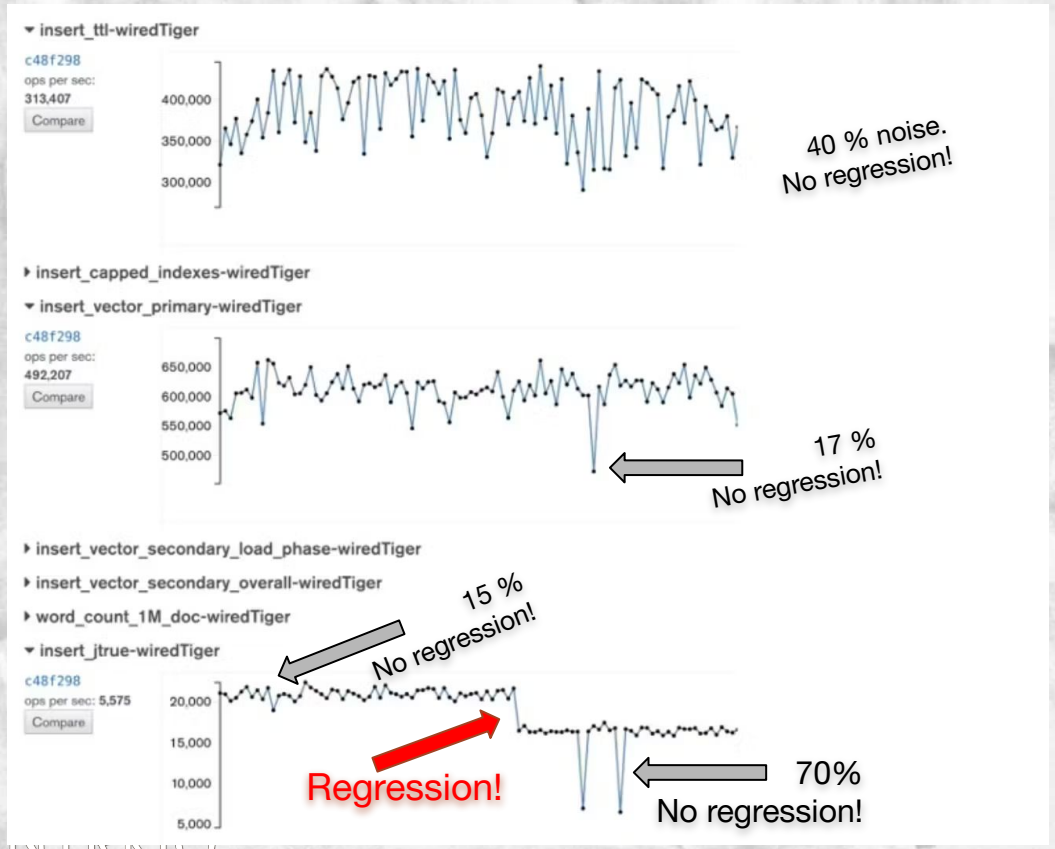
A DAY IN THE LIFE OF A MONGODB PERF ENGINEER, 2015.



A DAY IN THE LIFE OF A MONGODB PERF ENGINEER, 2015.



A DAY IN THE LIFE OF A MONGODB PERF ENGINEER, 2015.



VOCABULARY

Signal: The thing we are looking for

Noise: The thing we are not looking for

Outlier: Individual point that is clearly outside of the normal

Anomaly: Outlier

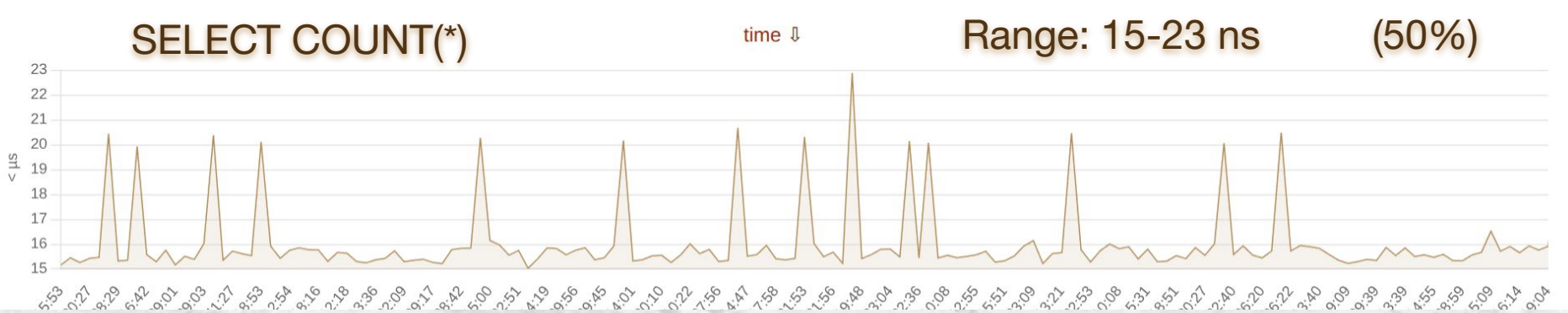
Change: A persistent change in the signal

Variance: Is a specific mathematical variable (which we rarely use)

Variability, range, noise: Use these instead!

EVERYONE ON GITHUB, 2025:

(TURSO DATABASE, GITHUB RUNNERS)

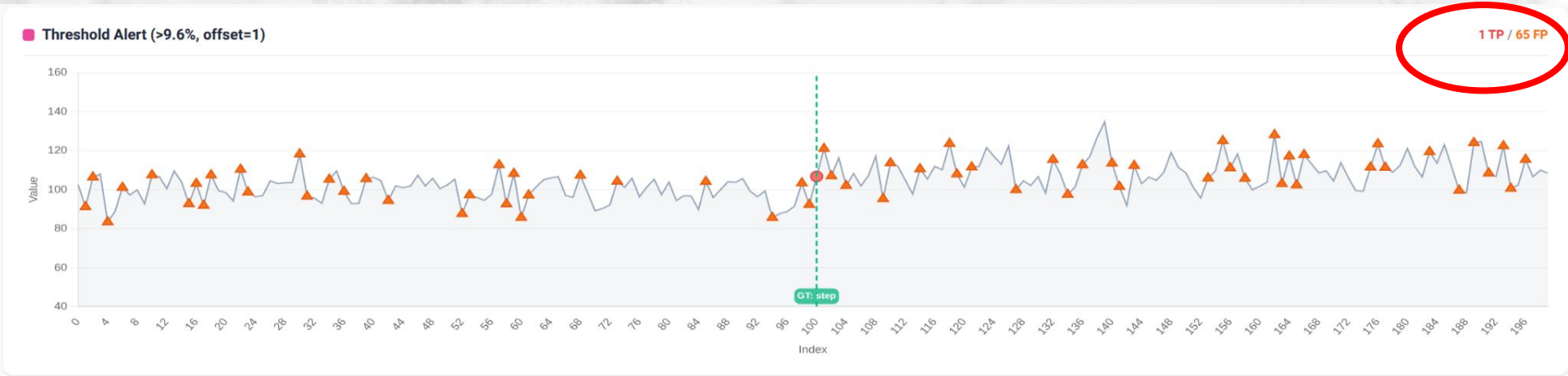


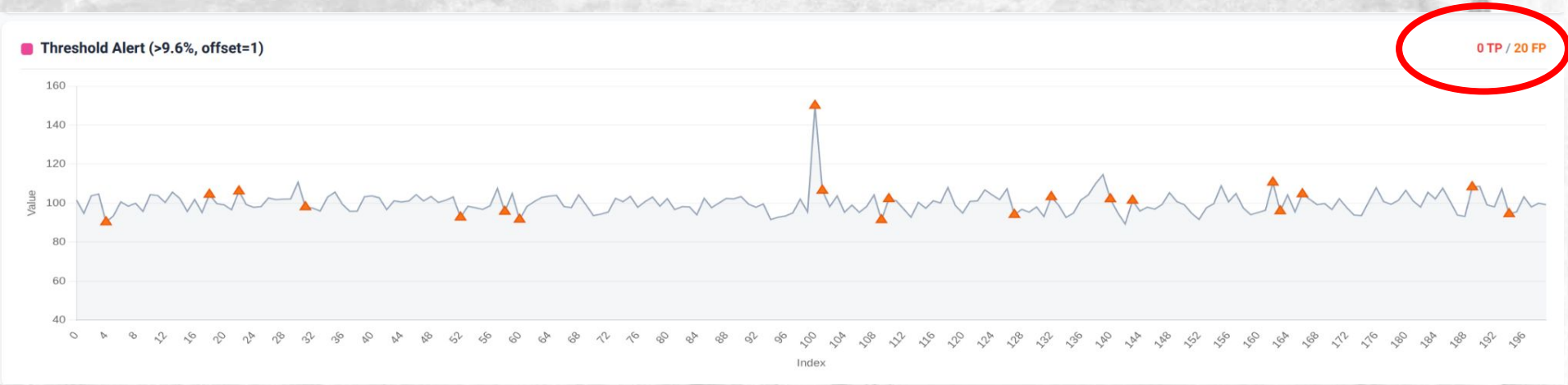
NYRKIÖ

AUTOMATION IS ONE OF THE CORE PRINCIPLES...

Threshold based alerting…

NYRKIÖ


Threshold Alert (>9.6%, offset=1)
1 TP / 65 FP
Value
Index
GT: step


Threshold Alert (>9.6%, offset=1)
0 TP / 20 FP
Value
Index

Core principles (for change point detection)

Math is hard*.

*) unfortunately math is also necessary

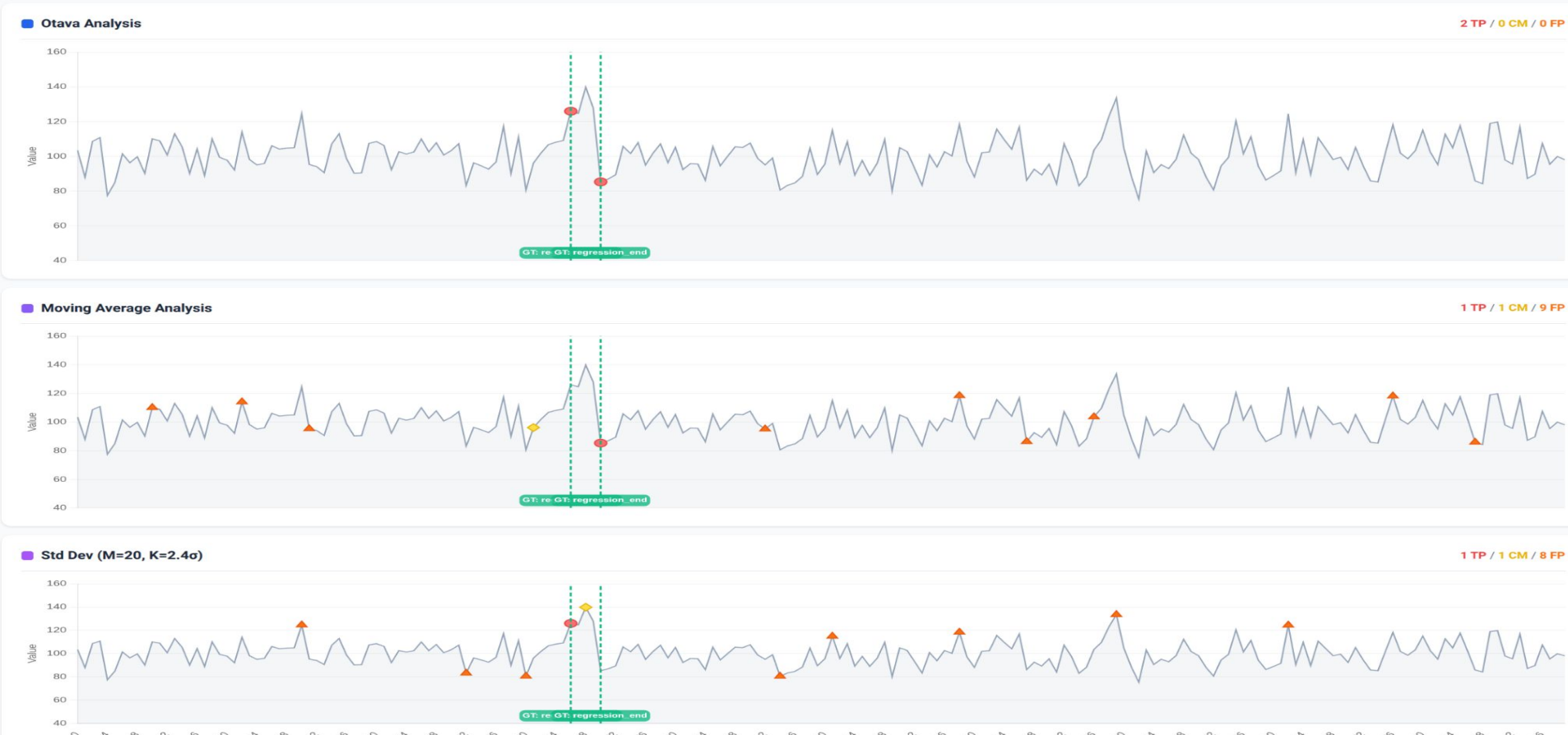
NYRKIÖ

E-DIVISIVE MEANS (MATTESON & JAMES, 2014)

A BIT MORE SOPHISTICATED COMPETITION

Demo app:
github.com/nyrkio/otava-test-data

OTAVA REALLY SHINES WITH 2 NEARBY POINTS



otava.apache.org

MINIMIZING NOISE IN BENCHMARKS

NYRKIÖ

Core principles (for server tuning)

Assume nothing. Measure everything.

Apply the scientific process

Don’t believe everything you read on the internet

NYRKIÖ

REASONS WHY BENCHMARK RESULTS ARE SO NOISY?

NYRKIÖ

REASONS WHY BENCHMARK RESULTS ARE SO NOISY?

CLOUD

NOISY NEIGHBOR

?

CLOUD

SOVEREIGN CLOUD

BAD PROGRAMMERS

Fun exercise: retroactively list assumptions built into your current architecture

Dedicated instance = more stable performance

Placement groups minimize network latency & variance

Different availability zones have different hardware

For write heavy tests, noise comes from disk

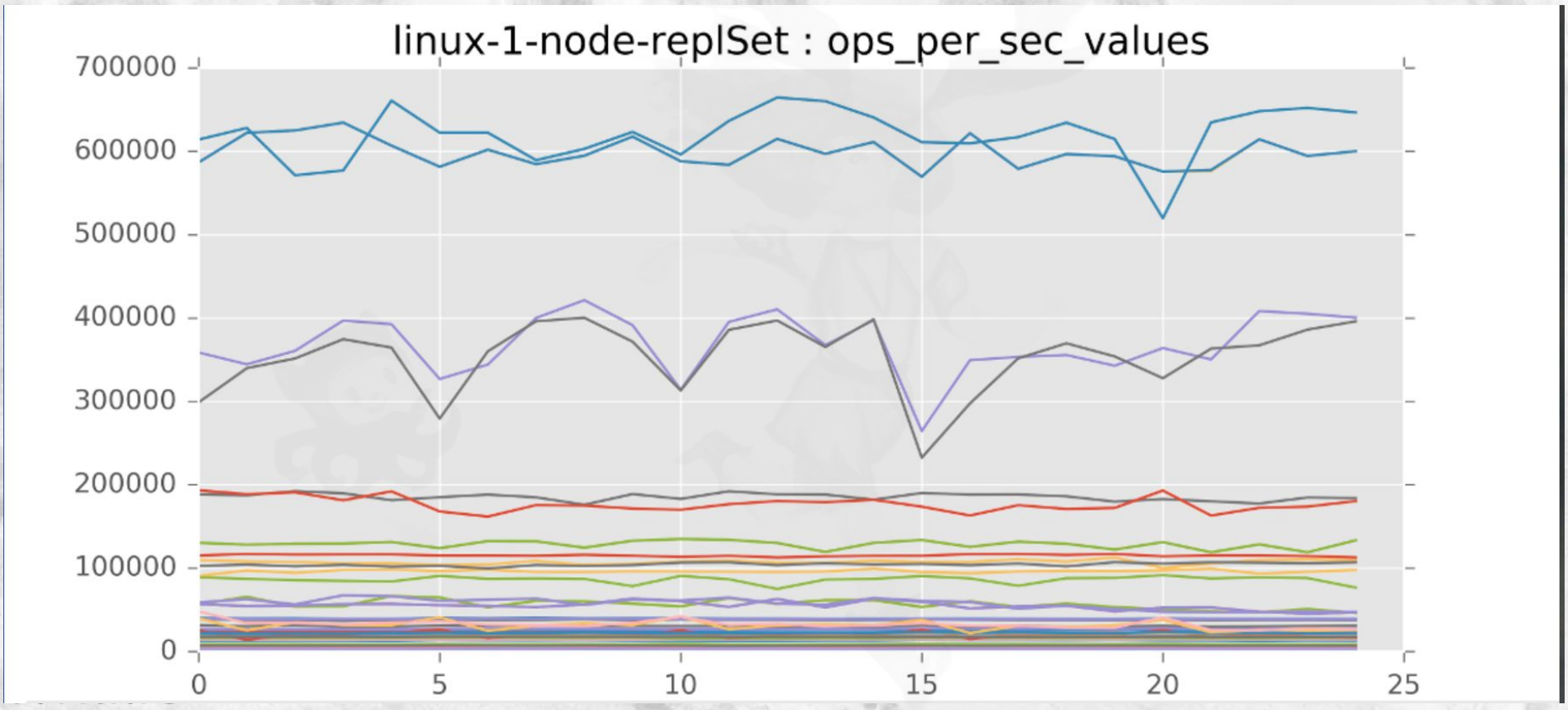
Ephemeral (SSD) disks have least variance

There are good and bad EC2 instances

Just use i2 instances (better SSD)

You can't use cloud for performance testing

1 MONGODB BINARY, 5 SERVERS, REPEAT TESTS 5X (2017)



Fun exercise: retroactively list assumptions built into your current architecture

Dedicated instance = more stable performance

Placement groups minimize network latency & variance

Different availability zones have different hardware

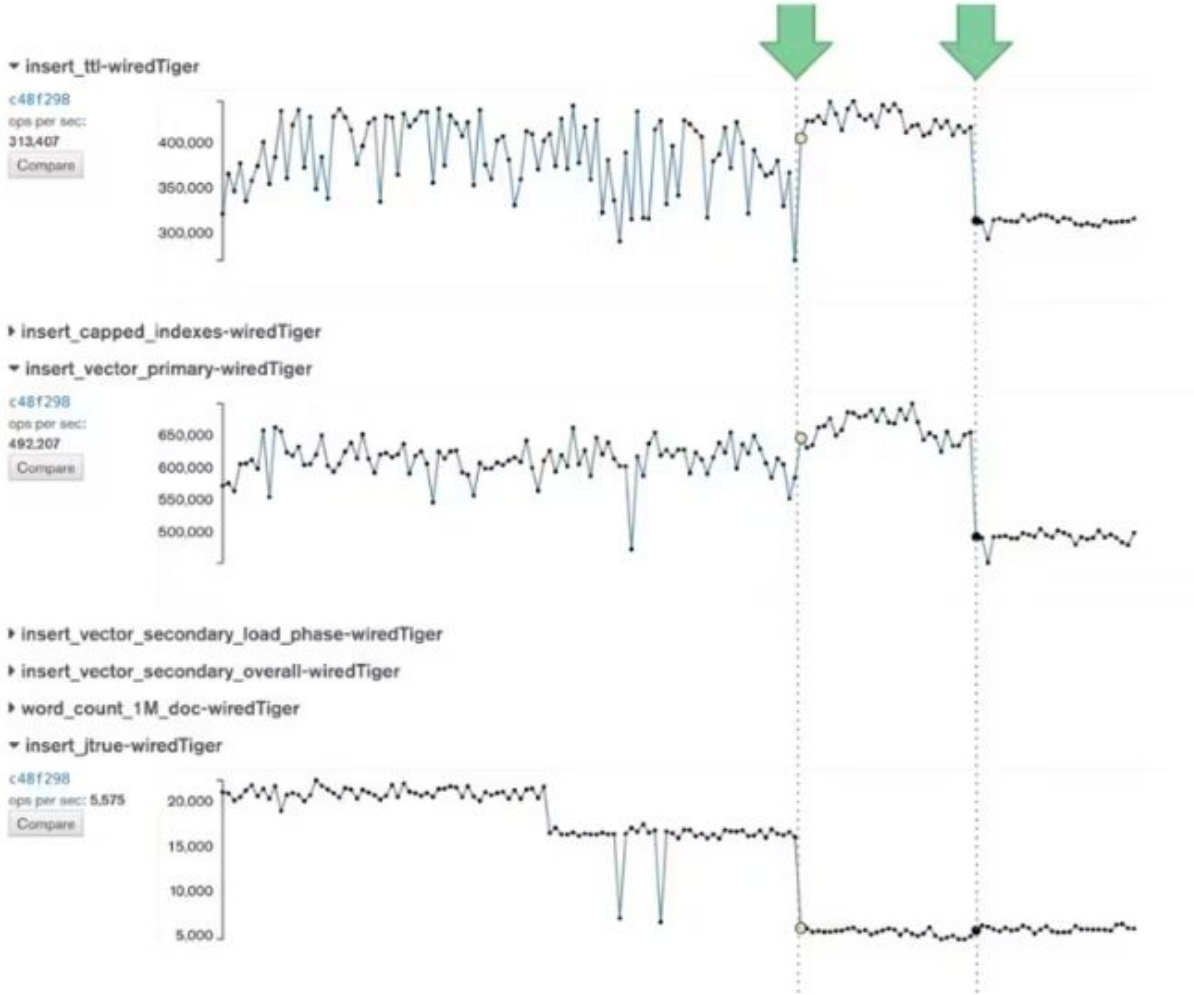
For write heavy tests, noise comes from disk

Ephemeral (SSD) disks have least variance

There are good and bad EC2 instances False

Just use i2 instances (better SSD)

You can't use cloud for performance testing



SSD -> EBS

CPU: No HT, single socket, scheduling

Noise range for all tests: 5%

WAS IT THE NEIGHBORS?

Continuous Benchmarking is hard because…

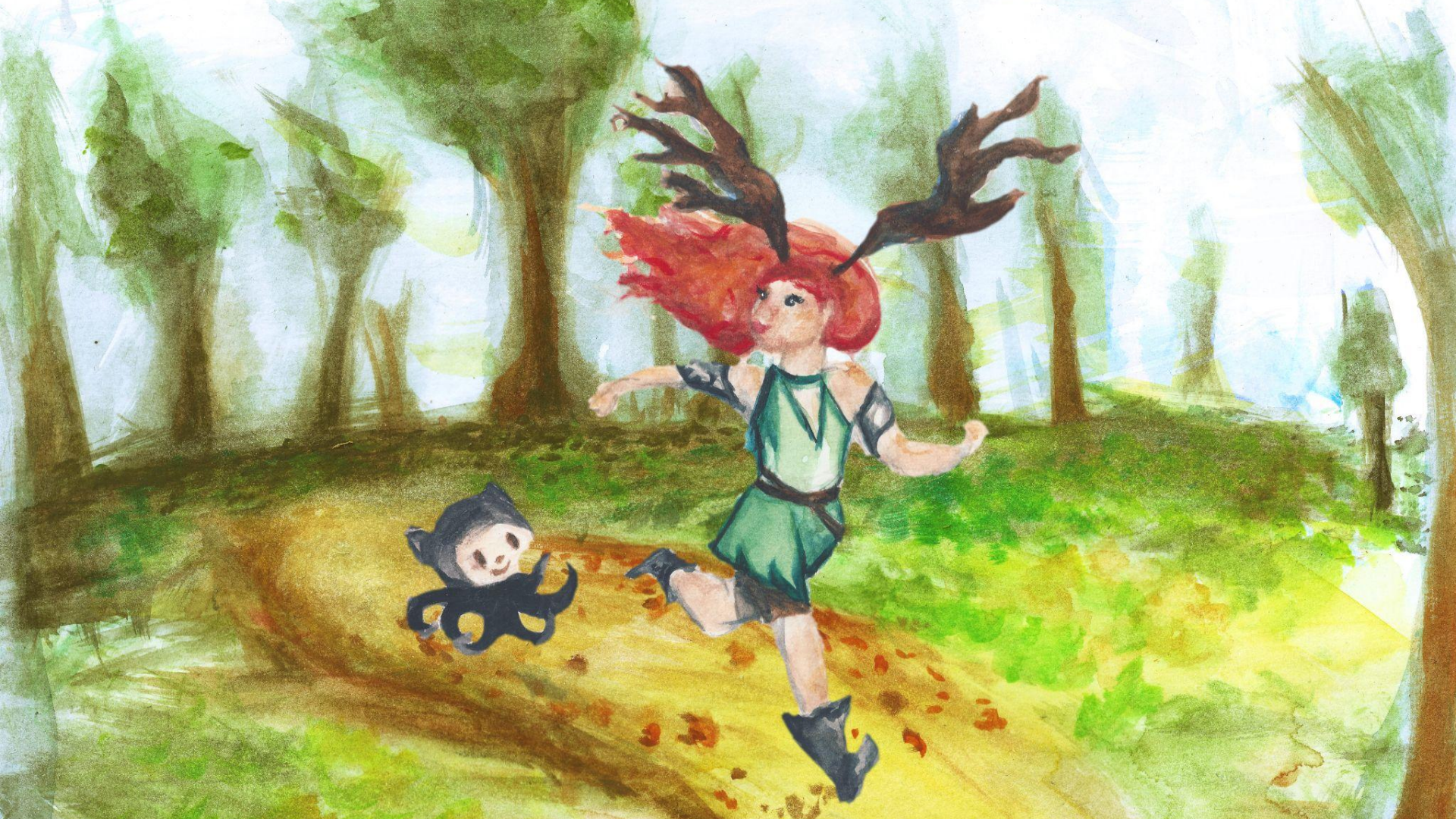
- Your hardware is actively working against you:
  - CPU Frequency scaling
  - CPU boost
  - HyperThreading…
  - NUMA architecture…

…and that was just the CPU!

```
man cpupower
```

NYRKIÖ

NYRKIÖ



NYRKIÖ GitHub Runners

NYRKIÖ GitHub Runners

1. Install as GitHub app: nyrkio.com
2. Pick a subscription (20 c/h)
3. In workflow.yml:

```
runs-on : nyrkio_2
```

C7a instances
Hand picked &
Carefully tuned

NOT best performance
NOT for price/performance

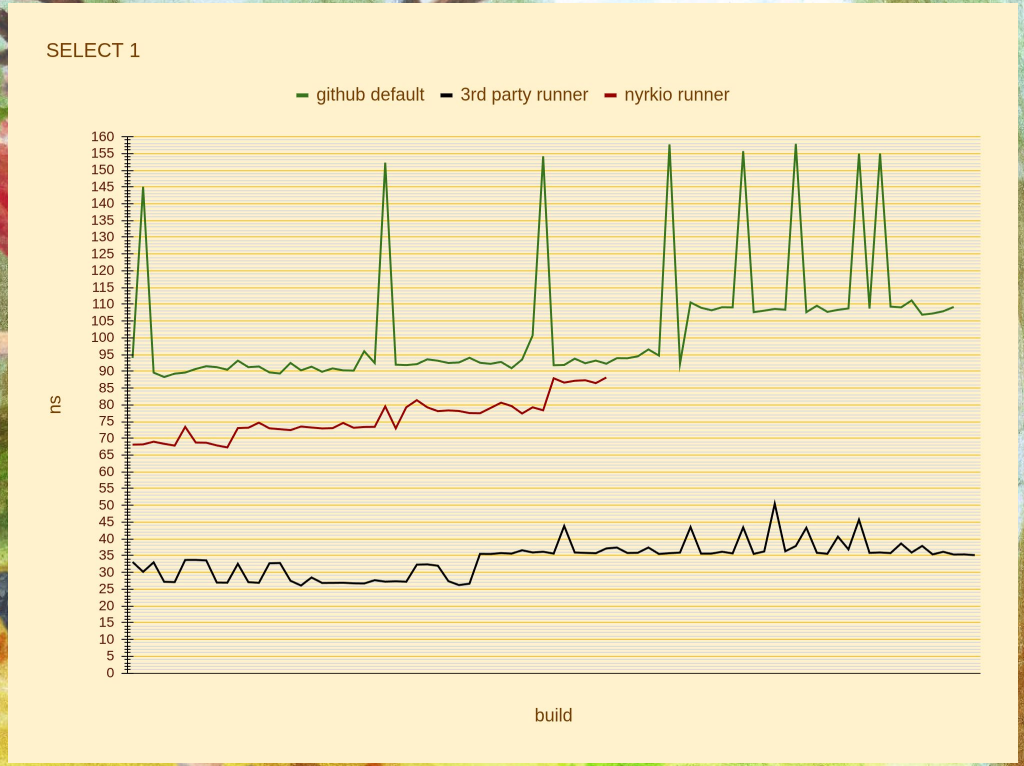
But

REPEATABLE
performance

NYRKIÖ GitHub Runners

min-max ranges (ns)

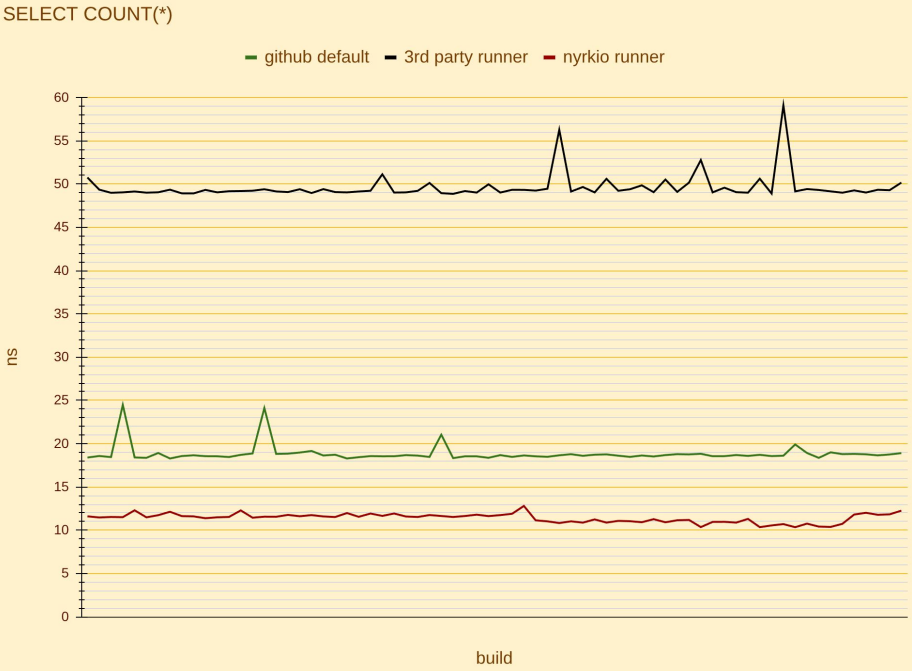
github default	3rd party runner	nyrkio runner
88	35	73
154	51	75
66 ns	15 ns	**2 ns**
75%	44%	3%



NYRKIÖ GitHub Runners

min-max ranges (ns)

github default	3rd party runner	nyrkio runner
18	49	11
25	59	13
6 ns	10 ns	**1 ns**
30%	21%	12%



WHAT IMPROVED IN 10 YEARS?

C7a family offers high fidelity performance for 100% of the price

- No hyperthreading
- Single CPU socket
- AMD?

Future opportunities:

- sub nano-second precision
- Dist-sys: Run K8s cluster, on 128 core server

WHAT HAVE WE LEARNED?

1. Start with a simple benchmark, Run ***continuously***

2. Apply

Math & Science

… until . false positives no longer hurt

3. Performance tuning for *repeatable* results is counter intuitive.

WHAT HAVE WE LEARNED?

1. Start with a simple benchmark, Run ***continuously***

2. Apply

Math & Science

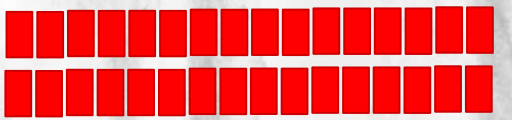
… until .
false positives
no longer hurt

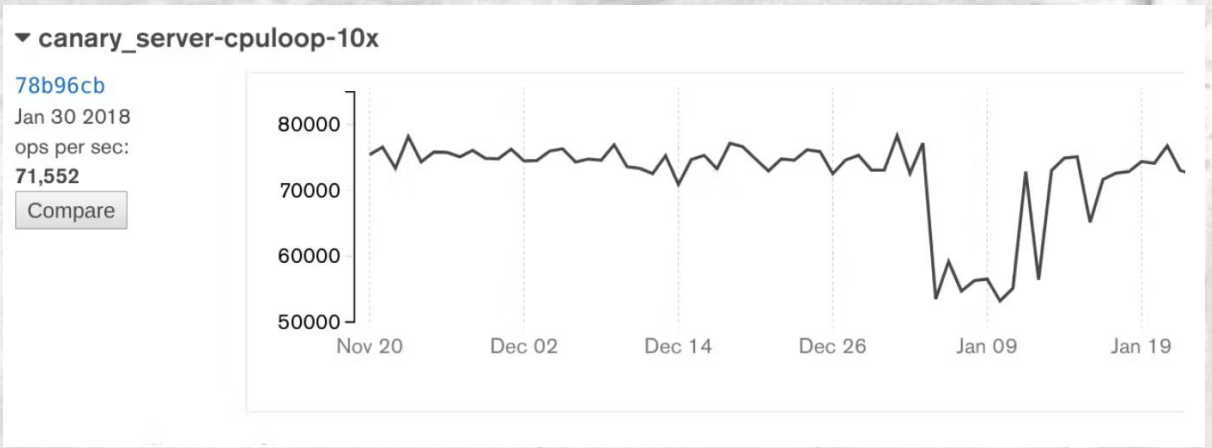
3. Performance tuning for *repeatable* results is counter intuitive.

PS: CANARIES

Add tests that measure your infrastructure.

- CPU
- Disk
- Network





NYRKIÖ

CREDITS AND REFERENCES

Nyyrikki and cat running, watercolor: Ebba Ingo

Velociraptor: Wikimedia commons

Otava test data graphs:
Joe Drumgoole & Claude

Everyone who contributed to Change Detection, now known as Apache Otava (incubating)

Ingo,Daly: Reducing Variability in Performance Tests on EC2
www.youtube.com/watch?v=3kHGZ7niHI4

David Daly et.al.: The Use of Change Point Detection to Identify Software Performance Regressions in a Continuous Integration System, 2020.

Fleming & Kołaczkowski: Hunter: Using Change Point Detection to Hunt for Performance Regressions, 2023.

Fixing Performance Regressions Before they Happen (Netflix)

8 Years of Optimizing Apache Otava: How disconnected open source developers took an algorithm from n^3 to constant time

Processor state control for Amazon EC2 Linux instances

NYRKIÖ